

Ultimate Placement Handbook

Git, GitHub & REST APIs

A Professional Interview Preparation Guide

Master the fundamentals to crack your next technical interview

Table of Contents

1. [Chapter 1: Mastering Git & GitHub](#)
2. [Chapter 2: Demystifying REST APIs](#)
3. [Final Command Cheat Sheet](#)

Chapter 1: Mastering Git & GitHub

1. Introduction to Version Control

Version Control is a system that manages and tracks changes in code so you can safely go back to earlier versions if needed. It acts as a time machine for your codebase.

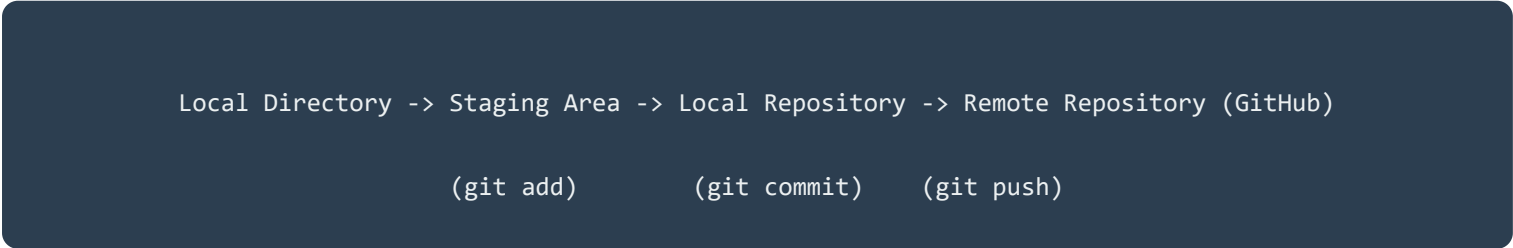
Git is a popular local version control tool installed on your computer that tracks these changes. **GitHub** is a cloud-based hosting service that stores your Git repositories online, allowing teams to collaborate.

Git vs GitHub Comparison

Feature	Git	GitHub
What is it?	Version control software on your local computer	Online platform to host Git repositories
Usage	Tracks changes and history locally	Enables global teamwork and code sharing
Internet	Works offline without internet	Requires internet connection

2. Core Concepts & Workflow

The standard workflow involves tracking changes locally and pushing them to a remote server like GitHub.



Setting up a Project

- `git config --global user.name "Your Name"` : Set your name.
- `git config --global user.email "you@example.com"` : Set your email.
- `git init` : Initializes a new local Git repository in the current folder.
- `git clone <url>` : Copies an existing repository from GitHub to your local machine.

Connecting to Remote

To connect your local project to a GitHub repository:

```
git remote add origin <repo-link>
```

Verify connection with: `git remote -v`

3. Tracking and Saving Changes

- `git status` : Shows what files have been changed, added, or deleted.
- `git add .` : Stages all modified files (prepares them for a commit).
- `git add filename.txt` : Stages only a specific file.
- `git commit -m "your message"` : Saves a snapshot of your staged files with a descriptive message.

Pushing and Pulling Code

- **First push:** `git push -u origin main` (sets up tracking for future pushes).
- **Subsequent pushes:** `git push`
- **Pulling latest code:** `git pull` (fetches and merges the latest changes from GitHub).
- **Force push:** `git push --force origin main` (DANGEROUS: overwrites remote history).

4. Branching and Merging

Branching allows you to create a separate copy of your code to work on features safely without breaking the `main` version.

- `git branch` : Lists all branches.
- `git checkout -b feature-name` : Creates a new branch and switches to it.
- `git checkout main` : Switches back to the main branch.

Merging Changes

Once your feature is complete, you merge it back to the main branch.

```
git checkout main
git merge feature-name
```

Pull Requests (PRs)

A Pull Request is a request to merge your branch into the main branch on GitHub. It allows the team to review the code, add comments, and approve before merging.

5. Undoing Mistakes

- `git restore filename` : Discards local uncommitted changes in a file.
- `git restore --staged file` : Unstages a file (removes it from the commit queue).
- `git reset --soft HEAD~1` : Undoes the last commit but keeps your changes staged.
- `git reset --mixed HEAD~1` : Undoes the last commit and unstages changes.
- `git revert <commit-id>` : Creates a new commit that undoes the changes of a previous commit.

6. Stashing and Fetching

Git Stash: Used when you need to switch branches but don't want to commit half-done work. It temporarily saves uncommitted changes (both staged and unstaged) to a clean working directory.

Command: `git stash` or `git stash push`

Git Pull vs Git Fetch

Command	Description
git fetch	Downloads new data from a remote repository into your local repository. It does not change your local working branch. You can review before merging.
git pull	A combination of <code>git fetch</code> + <code>git merge</code> . It downloads data and automatically updates your current branch.

7. Interview Q&A (Git)

Q: What is a commit?

A: A commit is a saved snapshot of your project files at a specific point in time.

Q: What is the .gitignore file?

A: A text file where you list files and directories you want Git to completely ignore (e.g., node_modules, passwords, large build files).

Q: What is HEAD in Git?

A: HEAD is a special reference pointer that indicates your current location in the repository's history ("Where am I right now?").

Q: Git Merge vs Git Rebase?

A: **Merge** preserves exactly what happened and when, creating a "merge commit". It is safe and does not alter existing commits. **Rebase** rewrites history by applying commits one by one onto another branch, giving a straight, linear history. It rewrites commit IDs, which can be dangerous if others have already pulled the branch.

8. Practical Scenarios

Scenario: You and a teammate edited the exact same line of code in the same file. What happens when you try to merge?

Action: You will get a **Merge Conflict**. Git cannot decide which version to keep. Git will mark the conflict areas in the file. You must open the file, manually decide which changes to keep, remove the conflict markers, save, stage (`git add`), and commit the resolved file.

9. Quick Revision Summary: Git

- **Init:** `git init` → `git add .` → `git commit -m "Msg"`
- **Link:** `git remote add origin <url>` → `git push -u origin main`

- **Daily:** `git add .` → `git commit` → `git push`
- **Branching:** Protects main code. Always create a branch for new features.
- **Merge Conflicts:** Happen when two people edit the same line. Requires manual resolution.

Chapter 2: Demystifying REST APIs

1. What is an API?

An **Application Programming Interface (API)** is a set of rules that allows different software systems to communicate with each other.

Examples: Frontend talking to Backend, Backend talking to a Database, or Backend talking to a Payment Gateway (like Stripe).

2. What is a REST API?

REST stands for **Representational State Transfer**. It is a set of architectural rules for designing APIs. REST APIs typically use the HTTP protocol, URLs, and JSON data to communicate between a client and a server.

Core Rules of REST:

- 1. **Use URLs as Nouns:** URLs represent resources (e.g., `/users`), not actions (avoid `/getUsers`).
- 2. **Use HTTP Methods as Verbs:** Use GET to read, POST to create, etc.
- 3. **JSON Responses:** Data is sent back as structured text (JSON) that all apps can understand.
- 4. **Statelessness:** Every request is independent. The server does not remember previous requests.

3. HTTP Methods and Status Codes

HTTP Methods

Method	Action	Example Endpoint
GET	Read data	<code>GET /users</code> (Get all users)

Method	Action	Example Endpoint
POST	Create new data	<code>POST /users</code> (Create a new user)
PUT	Replace all data	<code>PUT /users/42</code> (Replace user 42 entirely)
PATCH	Update part of data	<code>PATCH /users/42</code> (Update user 42's email)
DELETE	Remove data	<code>DELETE /users/42</code> (Delete user 42)

Common HTTP Status Codes

Status codes indicate if the request worked (2xx), if there was a client error (4xx), or a server error (5xx).

Code	Name	Meaning / When to use it
200 OK	Success	GET worked, data returned successfully.
201 Created	Created	POST worked, a new item was made.
204 No Content	No Content	DELETE worked, request done but no data to return.
400 Bad Request	Client Error	Your mistake. Invalid, wrong, or missing data sent by the client.
401 Unauthorized	Unauthenticated	Not logged in (missing or invalid auth token).
403 Forbidden	Unauthorized	User logged in, but lacks permission to access this resource.
404 Not Found	Not Found	Requested URL or resource does not exist.
500 Server Error	Server Fault	A bug or crash on the server side.

4. Request and Response Structure

The Request

A typical HTTP request contains:

- **Method + URI + Version:** e.g., `POST /users/42 HTTP/1.1`
- **Headers:** Extra metadata (e.g., `Host: api.example.com`, `Authorization: Bearer abc123`, `Content-Type: application/json`)
- **Body:** The data you are sending (used mostly in POST/PUT). Example: `{"name": "priya", "email": "p@gmail.com"}`

The Response

A typical HTTP response contains:

- **Status Code & Message:** e.g., `HTTP/1.1 200 OK`
- **Headers:** Metadata about the response (e.g., `Content-Type: application/json`)
- **Body:** The actual data returned. Example: `{"id": 42, "name": "priya"}`

5. Advanced API Concepts

Idempotency

Idempotency means sending the exact same request multiple times gives the same end result on the server.

- **Idempotent Methods:** GET, PUT, DELETE, HEAD, OPTIONS.

Example: If you `DELETE /user/42` once, the user is deleted. If you send it 10 more times, the end result is still the same (the user is deleted).

- **Non-Idempotent Methods:** POST. Sending the same POST request multiple times will create multiple duplicate resources.

Authentication vs Authorization

Concept	Meaning
Authentication	Verifying who the user is (e.g., logging in).

Concept	Meaning
Authorization	Checking what the user has access to do (permissions).

JWT (JSON Web Token): A standard token used securely for authentication and authorization in APIs.

Stateless vs Stateful

- **Stateless:** The server does not store client session state. Every request must contain all required information (like a JWT). REST APIs are fundamentally stateless.
- **Stateful:** The server stores client sessions/state. Future requests depend on previous ones.

Synchronous vs Asynchronous APIs

- **Synchronous API:** The client waits for the response before continuing. It is a blocking operation.
- **Asynchronous API:** The client does not wait. Other tasks can continue immediately, and the response is handled later when it arrives.

6. API Security & Best Practices

- **Rate Limiting:** A control mechanism that sets a maximum number of requests a user can make within a time period. It protects the API from being overloaded by too many calls and ensures fairness for all users.
- **Versioning:** Allows you to make changes to your API without breaking old users' code. Typically done in the URL (`/v1/users`) or headers. This ensures newer API versions can evolve while older versions remain stable.
- **CORS (Cross-Origin Resource Sharing):** A security mechanism that prevents unauthorized cross-origin requests. By default, a frontend running on one origin (domain) cannot access a backend on another origin unless the backend explicitly allows it via CORS headers.

7. Interview Q&A (APIs)

Q: Why do we use JSON in REST APIs?

A: JSON (JavaScript Object Notation) is lightweight, easy for humans to read, and universally

supported by almost all programming languages, making it perfect for data exchange.

Q: What is the difference between PUT and PATCH?

A: PUT replaces the entire resource. If you omit fields, they might be set to null. PATCH only updates the specific fields you send, leaving the rest of the resource intact.

8. Practical Scenarios

Scenario: You build an API and a user complains they are getting a 401 error, but occasionally they get a 403 error. What is the difference?

Action: 401 Unauthorized means the user hasn't provided a valid login token. They need to log in. 403 Forbidden means they are successfully logged in (authenticated), but they do not have admin rights or permissions to access that specific endpoint (authorization failure).

9. Quick Revision Summary: REST APIs

- **REST:** Architectural style using URLs (nouns) and HTTP methods (verbs).
- **Methods:** GET (Read), POST (Create), PUT (Replace), PATCH (Update), DELETE (Remove).
- **Status Codes:** 2xx (Success), 4xx (Client Error), 5xx (Server Error).
- **Idempotency:** Safe to repeat. (GET, PUT, DELETE). POST is NOT idempotent.
- **Stateless:** Server forgets you after each request; you must send your token every time.

Final Command Cheat Sheet

Git Core Commands

Action	Command
Initialize repo	<code>git init</code>
Clone repo	<code>git clone <url></code>
Check status	<code>git status</code>
Stage all changes	<code>git add .</code>
Commit changes	<code>git commit -m "msg"</code>
View history	<code>git log --oneline</code>
View visual history	<code>git log --graph --oneline</code>
See differences	<code>git diff</code>
Find who wrote a line	<code>git blame filename</code>

Git Branching & Remote

Action	Command
List branches	<code>git branch</code>

Action	Command
Create & switch branch	<code>git checkout -b name</code>
Merge branch	<code>git merge name</code>
Delete branch	<code>git branch -d name</code>
Add remote	<code>git remote add origin <url></code>
Push code	<code>git push origin main</code>
Pull code	<code>git pull origin main</code>
Fetch data only	<code>git fetch</code>